

Github: <https://github.com/movchun567/apz-kravchuk.git> , branch hazelcast_task2

1) Встановити і налаштувати Hazelcast

Hazelcast встановимо через докер, спочатку впевнимся що він є:

```
akravchuk@macA apz-kravchuk % docker version
Client:
 Version:           28.0.4
 API version:       1.48
 Go version:        go1.23.7
 Git commit:        b8034c0
 Built:             Tue Mar 25 15:06:09 2025
 OS/Arch:           darwin/amd64
 Context:           desktop-linux

Server: Docker Desktop 4.40.0 (187762)
Engine:
 Version:           28.0.4
 API version:       1.48 (minimum version 1.24)
 Go version:        go1.23.7
 Git commit:        6430e49
 Built:             Tue Mar 25 15:07:22 2025
 OS/Arch:           linux/amd64
 Experimental:      false
containerd:
 Version:           1.7.26
 GitCommit:         753481ec61c7c8955a23d6ff7bc8e4daed455734
runc:
 Version:           1.2.5
 GitCommit:         v1.2.5-0-g59923ef
docker-init:
 Version:           0.19.0
 GitCommit:         de40ad0
akravchuk@macA apz-kravchuk %
```

Докер є, далі створюємо нетворк:

```
akravchuk@macA apz-kravchuk % docker network create hazelcast-network
56dc68a5308f186f6a3db4de6522e97fd6717ef0617b9b0c7ff724eb03fa46f5
akravchuk@macA apz-kravchuk %
```

Перевіряємо чи нетворк створився:

```
akravchuk@macA apz-kravchuk % docker network ls
NETWORK ID      NAME                DRIVER             SCOPE
f34dbe1ecf11    apz-kravchuk_default bridge             local
c4a835286014    bridge             bridge             local
56dc68a5308f    hazelcast-network  bridge             local
5946ca7ce08f    host               host               local
12f9f40b82b0    hw1_default        bridge             local
c0c7ec8a5285    none               null               local
akravchuk@macA apz-kravchuk % docker network inspect hazelcast-network
[
  {
    "Name": "hazelcast-network",
    "Id": "56dc68a5308f186f6a3db4de6522e97fd6717ef0617b9b0c7ff724eb03fa46f5",
    "Created": "2026-03-01T17:00:09.979016942Z",
    "Scope": "local",
    "Driver": "bridge",
    "EnableIPv4": true,
    "EnableIPv6": false,
    "IPAM": {
      "Driver": "default",
      "Options": {},
      "Config": [
        {
          "Subnet": "172.20.0.0/16",
          "Gateway": "172.20.0.1"
        }
      ]
    },
    "Internal": false,
    "Attachable": false,
    "Ingress": false,
    "ConfigFrom": {
      "Network": ""
    },
    "ConfigOnly": false,
    "Containers": {},
    "Options": {},
    "Labels": {}
  }
]
akravchuk@macA apz-kravchuk %
```

Отже, ми запустили просту Hazelcast мережу за допомогою Docker.

2) Сконфігурувати і запустити 3 ноди (інстанси) об'єднані в кластер або як частину

Далі потрібно створити 3 ноди і об'єднати їх в кластер. Для цього запускаємо docker run з аргументами в яких зазначаємо назву ноди, до якої мережі вона належить, назву кластера,

порт та версію hazelcast. У кожній з нод буде відрізнятись лише назва та порт. Після цього перевіряємо чи в докері створився імедж ноди.

```
akravchuk@macA apz-kravchuk % docker run -d \
> --name hz1 \
> --network hazelcast-network \
> -e HZ_CLUSTERNAME=hw2 \
> -p 5701:5701 \
> hazelcast/hazelcast:5.3.8
592d7eb9c35efe3ae58fbf4b98f06df529fd92c8cb1ad96ad3b80ee3be74d714
akravchuk@macA apz-kravchuk % docker images | grep hazelcast
hazelcast/hazelcast 5.3.8 a95249a6fbe2 10 months ago 662MB
akravchuk@macA apz-kravchuk % docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
592d7eb9c35e	hazelcast/hazelcast:5.3.8	"hz start"	About a minute ago	Up About a minute	0.0.0.0:5701->5701/tcp	hz1

```
akravchuk@macA apz-kravchuk % docker run -d \
--name hz2 \
--network hazelcast-network \
-e HZ_CLUSTERNAME=hw2 \
-p 5702:5701 \
hazelcast/hazelcast:5.3.8
e7a45ea16d0b1aeccd8264d6c434f47dacd374ac235fe0840118c2c82d6f5ea4
akravchuk@macA apz-kravchuk % docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
e7a45ea16d0b	hazelcast/hazelcast:5.3.8	"hz start"	5 seconds ago	Up 4 seconds	0.0.0.0:5702->5701/tcp	hz2
592d7eb9c35e	hazelcast/hazelcast:5.3.8	"hz start"	2 minutes ago	Up 2 minutes	0.0.0.0:5701->5701/tcp	hz1

```
akravchuk@macA apz-kravchuk % docker run -d \
--name hz3 \
--network hazelcast-network \
-e HZ_CLUSTERNAME=hw2 \
-p 5703:5701 \
hazelcast/hazelcast:5.3.8
6737181f19baf8efee6a0b6b489ef6d5bf22f47e1156ac8d97299762731e838
akravchuk@macA apz-kravchuk % docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
6737181f19ba	hazelcast/hazelcast:5.3.8	"hz start"	9 seconds ago	Up 8 seconds	0.0.0.0:5703->5701/tcp	hz3
e7a45ea16d0b	hazelcast/hazelcast:5.3.8	"hz start"	49 seconds ago	Up 49 seconds	0.0.0.0:5702->5701/tcp	hz2
592d7eb9c35e	hazelcast/hazelcast:5.3.8	"hz start"	3 minutes ago	Up 3 minutes	0.0.0.0:5701->5701/tcp	hz1

За допомогою docker logs можна детально глянути, що саме відбувається в контейнері. В рядку Members можемо впевнитися, що кластер створився правильно.

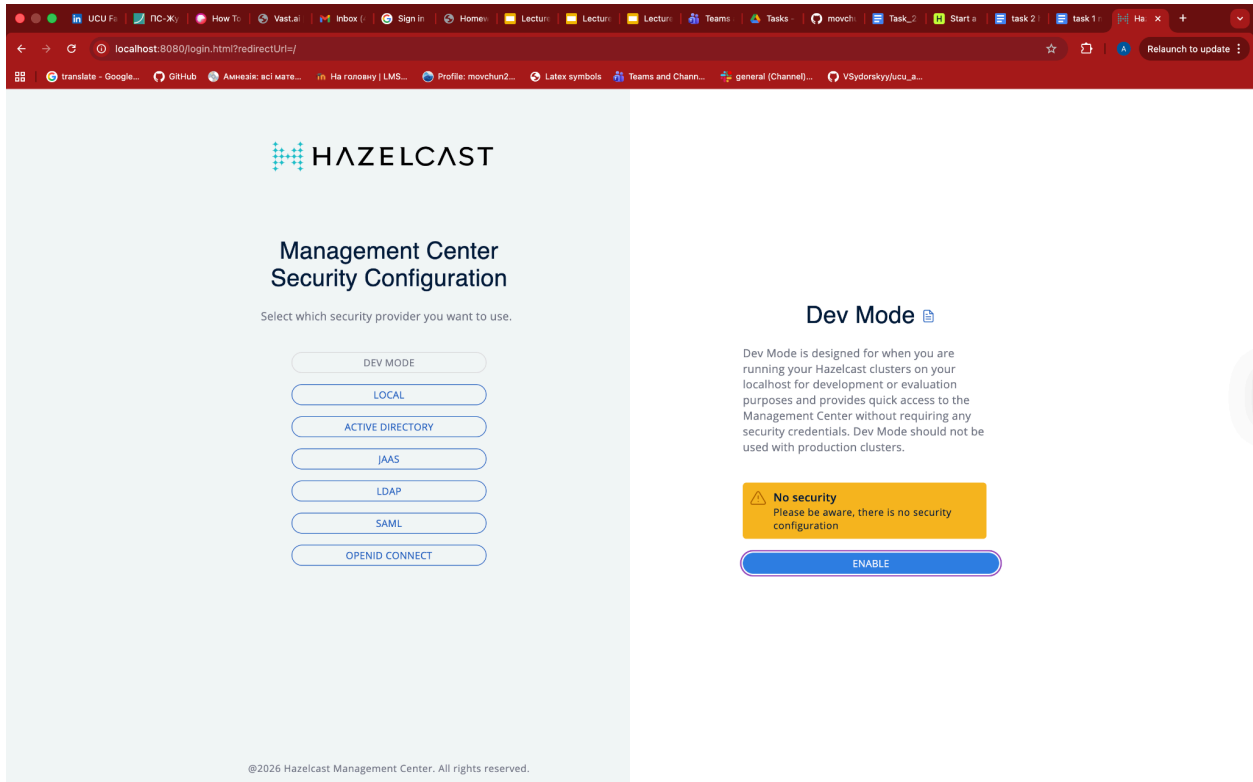
3) Продемонструйте роботу Distributed Map

В теорії, Distributed Map працює як HashMap в Java, але до нього мають доступ усі члени кластера. Має 271 partition, кожна з нод отримує свої. Для зручності завантажуюмо Management Center, щоб у веб форматі моніторити метрики і управляти кластером. Вказуємо порт, щоб потім відкрити в браузері.

```
akravchuk@macA apz-kravchuk % docker run -d \
  --name mc \
  --network hazelcast-network \
  -p 8080:8080 \
  hazelcast/management-center:5.3
Unable to find image 'hazelcast/management-center:5.3' locally
5.3: Pulling from hazelcast/management-center
4d5d1cbd7ece: Pull complete
c9034934f138: Pull complete
812782a8e062: Pull complete
08cf5234f468: Pull complete
4f4fb700ef54: Pull complete
369297ede937: Pull complete
929f704a3bdc: Pull complete
43ba685ca15c: Pull complete
Digest: sha256:5118eabc6cac1db058407d59ff66a8d45fe887bea415f734dd67189c3c415888
Status: Downloaded newer image for hazelcast/management-center:5.3
a142162eb3fd8af6191d9d9c2ab224bef5d7baa160d9114543b1c36a69ea8c0b
```

Перевіримо чи піднявся центр.

```
akravchuk@macA apz-kravchuk % docker ps | grep mc
a142162eb3fd hazelcast/management-center:5.3 "bash ./bin/mc-start..." 39 seconds ago Up 38 seconds 8081/tcp, 0.0.0.0:8080->8080/tcp, 8443/tcp mc
```



Центр піднявся, заходимо по порту який вказали, включаєм dev mode. Далі створюємо кластер, додаємо порти наших нод, а саме 172.20.0.2:5701, 172.20.0.3:5701, 172.20.0.2:5701 .



hw2



State: **ACTIVE**



Safe Members: **3/3**



Clients: **0**



Stream Processing and SQL
enabled

[VIEW CLUSTER](#)

Далі пишемо простий скрипт як в завданні на Python, робимо паралелізацію через `threadpool` задля швидкості. Скрипт який записує 1000 значень з ключами від 0 до 1000 називаємо `put_1000.py`.

```
akravchuk@macA apz-kravchuk % python3 put_1000.py
Connected. Clearing map...
Inserted: 1000
```

Map Statistics (In-Memory Format: BINARY)								
			RESET TIME	1 minute ago	→	now	Default View	
Member ^	Entries ^	Gets ^	Puts ^	Removals ^	Sets ^	Entry Memory ^	Events ^	
172.20.0.2:5701	321	0	1,000	0	0	41.66 kB	0	
172.20.0.3:5701	335	0	184	0	0	43.47 kB	0	
172.20.0.4:5701	344	0	165	0	0	44.64 kB	0	
TOTAL	1,000	0	1,349	0	0	129.78 kB	0	
1 - 3 of 3 Rows 10								

Очікуваний розподіл приблизно 33% на кожну з нод, щоб зрозуміти точні цифри треба звернути увагу на стовпчик `Entries`: `hz1` має 321 ключ, `hz2` має 335, `hz3` має 344. В сумі вийшло 1000.

Далі треба дослідити як зміниться розподіл, якщо Ноду виключаємо `docker stop <назва ноди>`
Виключити одну ноду:

Map Statistics (In-Memory Format: BINARY)

Member ^	Entries ^	
172.20.0.2:5701	485	
172.20.0.3:5701	515	
TOTAL	1,000	

Очікуваний результат 50% на кожну ноду, що ми й можемо спостерігати, 485 ключів у hz1 і 515 ключів у hz2.

Послідовно виключити 2 ноди:

Для цього спочатку запускаємо скрипт, а потім виключаємо одну ноду, а потім іншу, поки не лишиться одна.

Map Statistics (In-Memory Format: BINARY)

Member ^	^ Entries
172.20.0.2:5701	20
172.20.0.3:5701	17
172.20.0.4:5701	12
TOTAL	49

Поки доволі рівномірний розподіл, вирубаємо hz3.

Map Statistics (In-Memory Format: BINARY)

Member ^	^ Entries	
172.20.0.2:5701	74	
172.20.0.3:5701	39	
TOTAL	113	

Кластер побачив, що одна нода перестала відповідати, тому partition переходять в backup ноду. Чекаємо ще трошки і вирубаємо hz2.

Map Statistics (In-Memory Format: BINARY)

Member ^	^ Entries	
172.20.0.2:5701	1,000	
TOTAL	1,000	

Сталося як і в попередньому випадку, усі дані перейшли до backup ноди, і всі ключі зробилися.

Одночасно виключити 2 ноди:

В теорії, кластер не має встигнути резервно перекопіювати всі дані, тому просто вимикається.

Member ^	^ Entries
172.20.0.2:5701	168
TOTAL	168

```
akravchuk@macA apz-kravchuk % python3 put_1000.py
Connected. Clearing map...
Inserted: 168
```

По скрінах видно, що сталося як очікувалося, кластер не встиг переправити всі дані з вимкнених нод, тому вимкнувся.

- Чи буде втрата даних?

В першому випадку ні, бо кожний partition при вимкненні ноди до якої він приписаний, встигає перейти до backup ноди і зберегти дані. Втрати даних не буде при backup ≥ 1 . В другому випадку теж втрати даних не буде, бо partition завжди мають куди перейти. В третьому випадку з високою вірогідністю буде втрата даних, бо не вистачить часу на міграцію partitions, і partition мігрують в нікуди(якщо одна з backup нод вимикається).

- Яким чином зробити щоб не було втрати даних?

Для 3 нод зробити 2 backup, щоб завжди було куди переносити дані при аварії, але тоді одна надо без резервів. Persistence - дані записуються резервно на диск і дані не втрачаються. Зробити більше нод.

4) Продемонструйте роботу Distributed Map without locks

Кожен клієнт робить інкремент одного ключа як три операції: get key -> +1 -> put key. Це не атомарно, тому два клієнти можуть перезаписувати старі значення. Тому 30K в кінці не вийде. Скрипт написаний у файлі **map_no_locks.py**

```
akravchuk@macA apz-kravchuk % python3 map_no_locks.py
Starting
Clients: 3, iters each: 10000, expected: 30000
Final value: 11782
Time (sec): 33.799
```

Member ^	^ Entries	^ Gets
172.20.0.2:5701	0	0
172.20.0.3:5701	0	50,000
172.20.0.4:5701	1	163
TOTAL	1	50,163

(операцій більше, бо тестив)

Втратили близько 60% відсотків інкрементів, фінальне значення вийшло 11782.

5) Зробіть те саме з використанням песимістичним блокування та поміряйте час:

Схема як в no locks, але перед записом клієнт бере ключ, записує і потім звільняє комусь іншому. Тут операції атомарні, тому має вийти 30K. Скрипт використовує батчі по 1000 штук, щоб прискорити виконання. Скрипт називається **pessimistic_lock.py**.

```

akravchuk@macA apz-kravchuk % python3 pessimistic_lock.py
Starting
[worker 1] done 2000
[worker 0] done 2000
[worker 2] done 2000
[worker 1] done 4000
[worker 1] done 6000
[worker 0] done 4000
[worker 2] done 4000
[worker 1] done 8000
[worker 1] done 10000
[worker 0] done 6000
[worker 2] done 6000
[worker 0] done 8000
[worker 2] done 8000
[worker 0] done 10000
[worker 2] done 10000
Clients: 3, iters each: 10000, expected: 30000
Final value: 30000
Time (sec): 216.847

```

Map Statistics (In-Memory Format: BINARY)

RESET TIME

Member ^	^ Entries	^ Gets	^ Puts
172.20.0.2:5701	0	0	0
172.20.0.3:5701	1	0	0
172.20.0.4:5701	0	11	12
TOTAL	1	11	12

1 – 3 of 3

Як і очікувано, 30K операцій за 216 секунд.

6) Зробіть те саме з використанням оптимістичним блокуванням та поміряйте час::

Схожий механізм, але оптиміст не блокує наперед. Він читає старе значення, пробує compare and set, якщо хтось встиг змінити то повторює дію. Тут операції атомарні, тому має вийти 30K. Скрипт використовує батчі по 1000 штук, щоб прискорити виконання. Скрипт називається **optimistic_lock.py**.

```
akravchuk@macA apz-kravchuk % python3 optimistic_lock.py
Starting
[worker 0] done 2000
[worker 0] done 4000
[worker 0] done 6000
[worker 0] done 8000
[worker 0] done 10000
[worker 1] done 2000
[worker 2] done 2000
[worker 1] done 4000
[worker 2] done 4000
[worker 1] done 6000
[worker 2] done 6000
[worker 1] done 8000
[worker 2] done 8000
[worker 1] done 10000
[worker 2] done 10000
Clients: 3, iters each: 10000, expected: 30000
Final value: 30000
Time (sec): 60.565
```

Map Statistics (In-Memory Format: BINARY)					RESET TIME
Member ^	Entries ^	Gets ^	Puts ^		
172.20.0.2:5701	0	0	0		
172.20.0.3:5701	1	20	23		
172.20.0.4:5701	0	12	12		
TOTAL	1	32	35		

1 - 3 of 3

Як і очікувано, 30K за 60 секунд.

7) Порівняйте результати кожного з запусків

Як очікувалось, в реалізації без блокувань, втрачаємо близько 60% даних(дійшло 11782).

Песиміст і оптиміст мають однаковий результат 30K, тільки оптиміст втричі швидший (60 секунд проти 216 секунд). Із-за черги на lock, песиміст програє, а оптиміст виграє завдяки постійним повторам.

8) Робота з Bounded queue

Distributed queue це потокобезпечна черга, в якій декілька клієнтів можуть писати одночасно за FIFO(також зберігає порядок). Bounded queue додатково має максимальний розмір, 10 в нашому випадку.

Спочатку видаляєм контейнери, адже треба змінити конфіг і додати туди Bounded Queue. Назва конфігу **hazelcast-docker.xml**.

```
(.venv) akravchuk@macA apz-kravchuk % docker rm -f hz1 hz2 hz3 2>/dev/null
hz1
hz2
hz3
```

Далі перезапускаєм контейнери, але вже з новим конфігом.

```
(.venv) akravchuk@macA apz-kravchuk % docker run -d --name hz1 --network hazelcast-network -p 5701:5701 \
-v "$PWD/hazelcast-docker.xml:/opt/hazelcast/config/hazelcast-docker.xml" \
hazelcast/hazelcast:5.3.8
b81191391f20f1c340e1390c048b8fb4378b326d5f6941911119730faf6583dd
```

```
(.venv) akravchuk@macA apz-kravchuk % docker run -d --name hz2 --network hazelcast-network -p 5702:5701 \
-v "$PWD/hazelcast-docker.xml:/opt/hazelcast/config/hazelcast-docker.xml" \
hazelcast/hazelcast:5.3.8
995d04dc1d57b43b426888b460e728671b6af9b6d65bc0a28c1446ef8f1e0ce8
```

```
(.venv) akravchuk@macA apz-kravchuk % docker run -d --name hz3 --network hazelcast-network -p 5703:5701 \
-v "$PWD/hazelcast-docker.xml:/opt/hazelcast/config/hazelcast-docker.xml" \
hazelcast/hazelcast:5.3.8
ababc6dcca92e1ef79133c68a0938b791c671a34a3415c5ba9c27c0cda9308b4
```

Далі пишемо скрипт з чергою під назвою **bounded_queue_test.py** . Один клієнт буде продюсером(писати в чергу), інші два консюмерами(читають з черги). Кожен елемент буде виданий одному клієнту за FIFO. Якщо черга пуста, консюмери блокуються. Коли продюсер робить put, то один з консюмерів прокидається і забирає собі число.

```
[P] put 90
[P] put 91
[C1] got 91
[C2] got 92
[P] put 92
[C1] got 93
[P] put 93
[C2] got 94
[P] put 94
[P] put 95
[C1] got 95
[P] put 96
[C2] got 96
[C1] got 97
[P] put 97
[P] put 98
[C2] got 98
[P] put 99
[C1] got 99
[C2] got 100
[P] put 100
```


Якщо черга буде заповнена, і ніхто не буде читати, то продюсер не зможе записати туди 11-й елемент.